

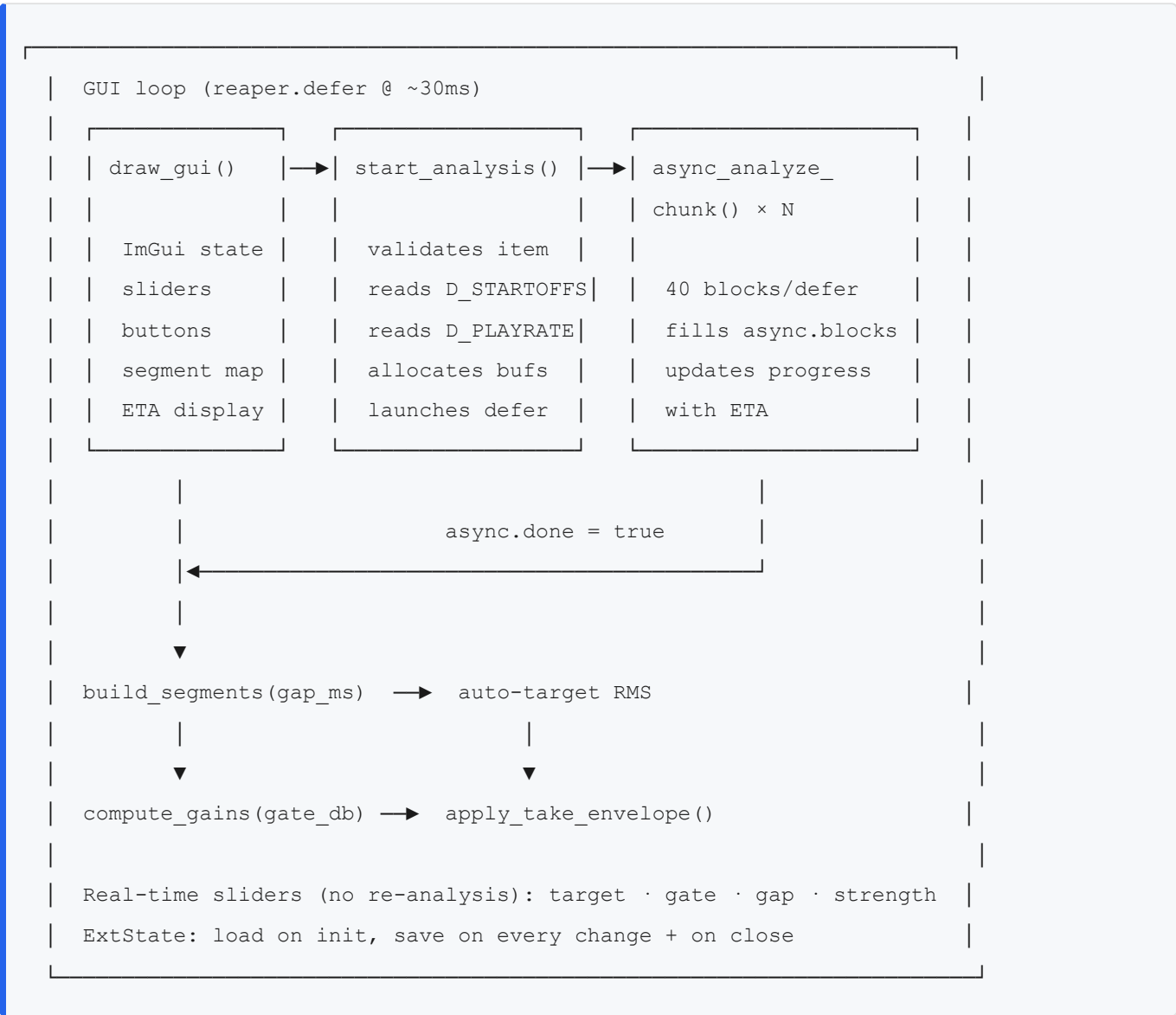
Vocal Volume Macro — Technical Documentation

Internal architecture, DSP algorithms, data flow, and REAPER API notes
Reflects `VocalVolumeMacro_v6.lua` — current production version

Table of Contents

1. Architecture overview
 2. Configuration table
 3. Signal processing pipeline
 4. Block analysis — `analyze_block`
 5. Async analysis loop
 6. Source time synchronisation
 7. Classification and smoothing
 8. Adaptive silence threshold
 9. Segmentation — `build_segments`
 10. Gain computation — `compute_gains`
 11. Auto-target RMS
 12. Envelope writing — `apply_take_envelope`
 13. Cache system
 14. Settings persistence — `ExtState`
 15. GUI loop and state machine
 16. Known limitations
 17. REAPER API reference
-

1. Architecture overview



All state lives in three module-level tables:

Table	Purpose
CFG	Static defaults, updated from GUI before each analysis
async	Mutable state shared between start_analysis and async_analyze_chunk
gui	GUI state, loaded from ExtState on startup

Math functions are localised at module level (local sin = math.sin etc.) to avoid repeated global table lookups. On LuaJIT (REAPER's runtime), this yields a ~15–20% speedup on the inner DFT loops.

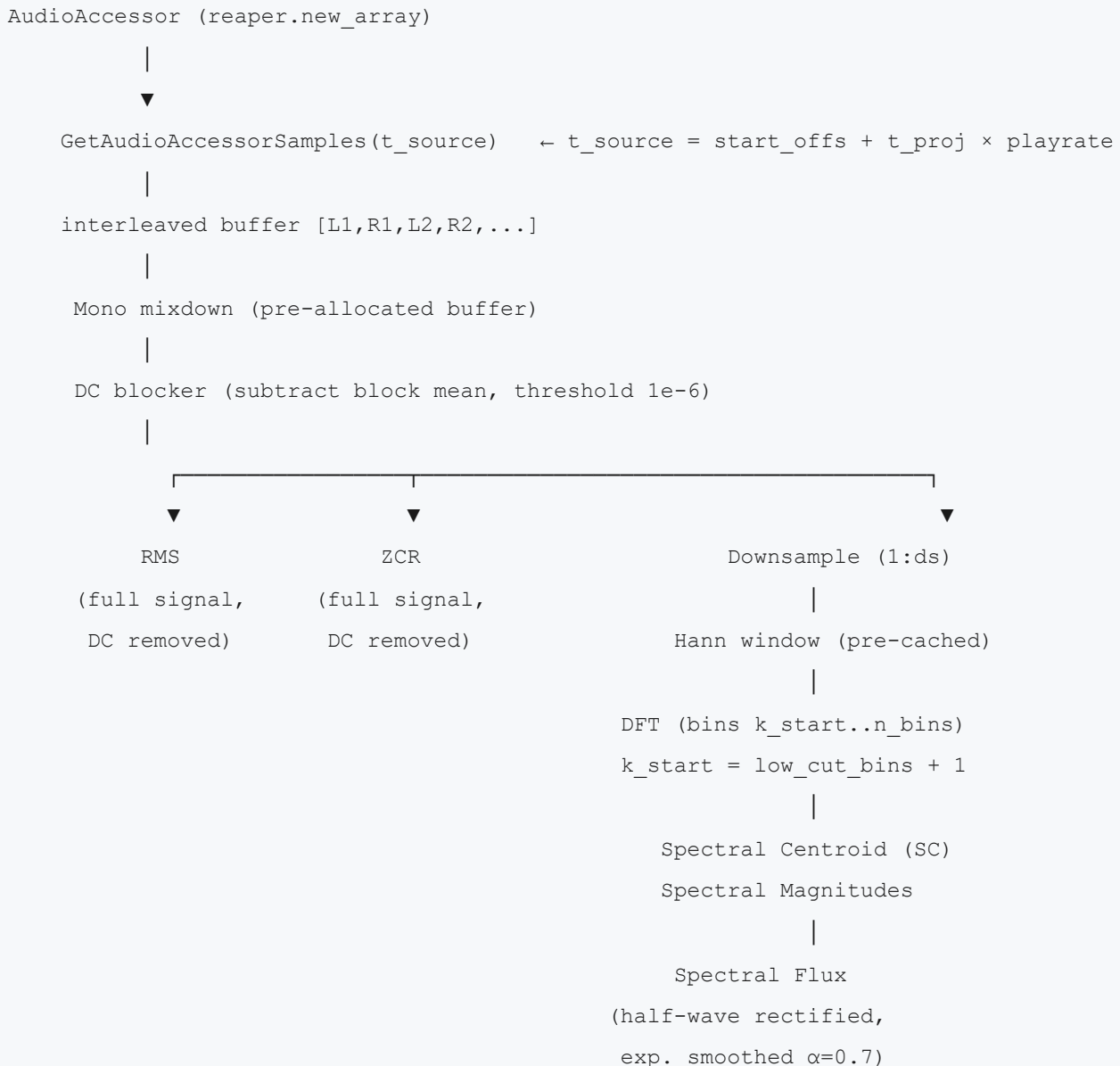
2. Configuration table

```
local CFG = {
    block_ms      = 10,      -- block duration (ms)
    min_silence_db = -50,    -- static floor (overridden by adaptive threshold)
    zcr_tonal_max  = 0.15,    -- ZCR threshold: tonal if below
    sc_tonal_max   = 2500,    -- spectral centroid threshold (Hz): tonal if below
    flux_onset_thresh = 0.30, -- spectral flux threshold: tonal if below
    smooth_frames  = 3,      -- temporal smoothing window (±N blocks)
    fade_ms        = 5,      -- transition fade duration (ms)
    min_seg_ms     = 30,     -- minimum segment duration before discard (ms)
    dft_downsample = 4,      -- downsampling factor for DFT
    n_bins         = 50,     -- DFT bins analysed
    low_cut_bins   = 2,      -- DFT bins skipped at low frequency (free low-cut filter)
    -- at ds=4, sr=44100: each bin ≈ 99 Hz
    -- low_cut_bins=2 → ignores everything below ~200 Hz
    flux_alpha     = 0.7,    -- exponential smoothing coefficient for spectral flux
    per_block_gain = false,  -- false = segment gain (Melodyne), true = Volume Rider
}
```

v6 GUI parameters (persisted via ExtState)

Parameter	Default	Real-time?
<code>zcr_thresh</code>	0.15	No — requires re-analysis
<code>sc_thresh</code>	2500 Hz	No — requires re-analysis
<code>flux_thresh</code>	0.30	No — requires re-analysis
<code>block_ms</code>	10 ms	No — requires re-analysis
<code>target_db</code>	auto	Yes
<code>gate_db</code>	-40 dB	Yes
<code>gap_ms</code>	100 ms	Yes — re-segments without re-analysis
<code>macro_pct</code>	80%	Yes
<code>fade_ms</code>	5 ms	Yes
<code>per_block</code>	false	Yes
<code>smooth_alpha</code>	0.25	Yes

3. Signal processing pipeline



Critical ordering: RMS and ZCR are computed on the raw mono signal **before** windowing. The Hann window reduces signal energy by ~4.26 dB — computing RMS after windowing would cause a systematic underestimation of block levels.

Low-cut filter: The DFT loop starts at `k_start = low_cut_bins + 1` rather than `k = 1`. This discards the lowest frequency bins (DC offset, sub-bass rumble, mic handling noise) from the spectral centroid calculation. Without this, a strong room rumble during a sibilant could pull the centroid low and misclassify an "S" as tonal.

4. Block analysis — `analyze_block`

Signature:

```

local rms, zcr, sc, mags, sum_mag = analyze_block(
    buf, block_samples, nch, samplerate,
    n_bins, ds, N_eff, trig, hann,
    mono, ds_buf, mags,    -- pre-allocated reusable buffers
    low_cut_bins          -- DFT low-cut (default 2)
)

```

The last four parameters are pre-allocated in `start_analysis` and reused across all blocks, avoiding ~5400 table allocations and GC cycles for a 3-minute take.

4.1 Mono mixdown — interleaved buffer

`GetAudioAccessorSamples` fills the buffer in **interleaved** format:

```

buf = { L1, R1, L2, R2, ..., Ln, Rn }

```

Index for sample `i`, channel `c` (0-based): `buf[(i-1) * nch + c + 1]`

Critical: `reaper.new_array(n)` is mandatory — a plain Lua table `{}` returns silence or crashes REAPER with a memory violation.

4.2 DC blocker

```

local dc = sum(mono) / block_samples
if abs(dc) > 1e-6 then
    for i = 1, block_samples do mono[i] = mono[i] - dc end
end

```

A DC offset shifts the signal baseline away from zero. The ZCR counts zero-axis crossings — a shifted signal crosses less often, causing artificially low ZCR values that misclassify noisy blocks as tonal. The `1e-6` threshold skips the correction on perfectly centred signals.

4.3 RMS

```

RMS = sqrt( Σ(mono[i]2) / N )

```

Computed on the full DC-corrected block. Not affected by the Hann window.

4.4 ZCR (Zero Crossing Rate)

```

ZCR = count(sign(mono[i]) ≠ sign(mono[i-1])) / (N - 1)

```

Normalised to [0, 1]. Voiced vowels: ~0.05–0.15. White noise: ~0.5.

4.5 Downsampling + Hann windowing

```
for i = 1, block_samples, ds do
  idx = idx + 1
  ds_buf[idx] = mono[i] * hann[idx]
end
```

Factor `ds = 4` reduces the DFT input from ~441 to ~111 samples. The Hann window is applied in the same pass.

N_eff — the off-by-one fix:

```
N_eff = floor((block_samples - 1) / ds) + 1
```

The naive `floor(N/ds)` gives $N-1$ for some values, making the trig tables one element too short → `nil` arithmetic at the last sample → analysis crash.

Hann window:

```
w[n] = 0.5 * (1 - cos(2π(n-1) / (N_eff-1)))
```

Eliminates spectral leakage. Without it, the spectral centroid error is ±15–30%. With Hann: ±2%. Pre-computed once per `N_eff`, cached in `hann_cache`.

4.6 Partial DFT with low-cut

```
local k_start = (low_cut_bins or 2) + 1

for k = k_start, n_bins do
  re = Σ ds_buf[n] * cos(2πk(n-1)/N_eff)
  im = Σ ds_buf[n] * (-sin(2πk(n-1)/N_eff))
  mags[k] = sqrt(re² + im²)
end
```

Skipping the first `low_cut_bins` bins removes the DC component and sub-bass from the centroid calculation. Cost: zero (fewer iterations, not extra ones).

All `cos`/`sin` values are pre-computed in `trig.cos[k][n]` and `trig.sin[k][n]`, cached per `(N_eff, n_bins)`.

Frequency resolution:

```
freq_step = (samplerate / ds) / N_eff
freq_max  = n_bins * freq_step
```

4.7 Spectral Centroid

```
SC =  $\Sigma(\text{mags}[k] \times \text{freq}[k]) / \Sigma(\text{mags}[k])$ 
```

Protected against division by zero: `(sum_mag > 0) and (sum_weighted / sum_mag) or 0`.

5. Async analysis loop

Chunked execution

```
local BLOCKS_PER_DEFER = 40

local function async_analyze_chunk()
  -- process up to 40 blocks per defer frame
  -- if not done: reaper.defer(async_analyze_chunk)
  -- if done:      async.done = true
end
```

At 44100 Hz / 10 ms blocks: 40 blocks = 400 ms of audio per defer frame (~30 ms wall-clock). A 3-minute take (~18 000 blocks) completes in ~450 frames ≈ 13 s elapsed. REAPER stays fully responsive throughout.

ETA estimation

```
local elapsed  = r.time_precise() - gui.analysis_start
local total_est = elapsed / async.progress
local remaining = total_est - elapsed
```

Displayed only after 2% progress to avoid wild early estimates.

pcall safety

The entire chunk body runs inside a `pcall`. Any unhandled error (nil arithmetic, API failure) is caught, the AudioAccessor is destroyed, and a clear error message appears in both the console and a popup.

Double-free protection: `async.aa` is set to `nil` immediately after `DestroyAudioAccessor`. The error handler checks `if async.aa` before destroying, preventing a second call on an already-freed object.

Window close cleanup

If the GUI window is closed while an analysis is running, the `loop()` function directly destroys the accessor rather than relying on `async.cancelled` — because once the defer loop stops, no further chunk will ever run:

```
if async.aa then
    r.DestroyAudioAccessor(async.aa)

    async.aa      = nil
    async.running = false
end

-- Note: r.ImGui_DestroyContext(ctx) must NOT be called in ReaImGui.
-- The context is managed by Lua's GC automatically.
```

6. Source time synchronisation

The problem

`GetAudioAccessorSamples` reads raw source file data — time is in **source space**. If an item is a trim of a longer recording (e.g. the visible portion starts 30 s into the file), `t_start = b * bs / sr` always begins at 0.0 s of the source — the wrong audio is analysed, and the automation is placed at the wrong position.

The fix — three locations

`start_analysis` — read `D_STARTOFFS` and `D_PLAYRATE`, inject into `async`:

```
local start_offs = r.GetMediaItemTakeInfo_Value(take, "D_STARTOFFS")
local playrate   = r.GetMediaItemTakeInfo_Value(take, "D_PLAYRATE")
if playrate == 0 then playrate = 1.0 end

async.start_offs = start_offs
async.playrate   = playrate
```

`async_analyze_chunk` — compute `t_source` and pass it to `GetAudioAccessorSamples`:


```

local t_proj    = b * bs / sr                -- relative to item start
    local t_source = start_offs + (t_proj * playrate)    -- position in source file

    r.GetAudioAccessorSamples(aa, sr, nch, t_source, bs, buf)

    blocks[b + 1] = {
        time = t_source,    -- stored directly — ready for InsertEnvelopePoint
        ...
    }

```

`apply_take_envelope` — `blocks[i].time` already contains the correct source coordinate; no recalculation:

```

local env_t = blocks[i].time    -- already t_source
    r.InsertEnvelopePoint(env, env_t, to_env(g_cur), ...)

```

The Take Volume Envelope API expects times in source space — which is exactly what `t_source` provides.

7. Classification and smoothing

Raw classification

A block is tonal only if **all four** conditions hold simultaneously:

```

audible  = rms_db > adaptive_silence    -- above noise floor
low_zcr  = zcr    < cfg.zcr_tonal_max   -- periodic signal
low_sc   = sc     < cfg.sc_tonal_max     -- energy in low frequencies
no_onset = flux   < cfg.flux_onset_thresh -- no abrupt spectral attack

```

Spectral flux

```
flux = 0
for k = 1, n_bins do
  diff = mags[k] - prev_mags[k]
  if diff > 0 then flux = flux + diff end  -- half-wave rectification
end
flux = flux / sum_mag  -- normalise by total energy

flux_smooth = alpha * flux_smooth + (1 - alpha) * flux  --  $\alpha = 0.7$ 
```

Half-wave rectification captures energy increases only — characteristic of consonant onsets. The exponential smoother prevents single-frame spikes from triggering false detections.

Temporal smoothing — strict majority vote

```
for i = 1, #blocks do
  votes = 0
  for j = max(1, i - sf), min(N, i + sf) do
    votes += raw_tonal[j] ? 1 : 0
  end
  blocks[i].is_tonal = (votes / total) > 0.5  -- strict majority
end
```

Window: ± 3 blocks (70 ms at 10 ms/block).

Why `> 0.5` and not `>= 0.5`: with a 3-block window, a single isolated tonal block scores $1/3 \approx 0.33$ — safely below both thresholds. But if `smooth_frames` is reduced to 1 (3-block window), a single tonal block could score $1/3 \approx 0.33$ and pass with `>=`. The strict `>` guarantees a block needs at least one tonal neighbour to survive, preventing micro-glitches that would boost breath noise for a few milliseconds.

8. Adaptive silence threshold

After all blocks are analysed:

```
table.sort(rms_values)
local p10_idx      = max(1, floor(N * 0.10))
local noise_rms    = rms_values[p10_idx]
local noise_db     = lin_to_db(noise_rms)
local adaptive_silence = clamp(noise_db + 6, -60, -30)
```

Why percentile 10: the quietest 10% of blocks represent room noise, breath gaps, and pre-roll silence. Using the minimum would be fragile against a few anomalous blocks.

+6 dB margin: blocks only slightly above the noise floor (residual breath noise) are excluded from tonal classification.

Clamped to [-60, -30]: prevents unreasonable thresholds in extreme cases.

Logged to console:

```
[VVM] Noise floor estimate: -52.3 dB → adaptive silence threshold: -46.3 dB
```

9. Segmentation — `build_segments`

Signature:

```
local segments = build_segments(blocks, min_seg_ms, block_ms, max_gap_ms)
```

`max_gap_ms` is the user-adjustable gap bridging tolerance (default 100 ms). When changed in the GUI, segments are rebuilt immediately without re-analysis.

Three-pass algorithm

Pass 1 — raw tonal groups:

Consecutive blocks with `is_tonal = true` are grouped into spans `{from, to}`. This produces a list of raw tonal segments separated by non-tonal gaps of varying length.

Pass 2 — Gap Bridging (vibrato fix):

```

local max_gap_blks = ceil(max_gap_ms / block_ms)

local current = raw[1]
for i = 2, #raw do
    local gap = raw[i].from - current.to
    if gap <= max_gap_blks then
        -- Short gap: vibrato dip or brief consonant — bridge it
        for b = current.to + 1, raw[i].from - 1 do
            blocks[b].is_tonal = true    -- force bridged blocks tonal
        end
        current.to = raw[i].to          -- extend current segment
    else
        -- Long gap: genuine word boundary
        merged[#merged + 1] = current
        current = raw[i]
    end
end
end

```

Why bridging is needed: a natural vibrato oscillates at 5–7 Hz, creating amplitude dips of ~70–100 ms. Without bridging, these dips produce `is_tonal = false` blocks, which `build_segments` turns into gaps between segments. The result is a sawtooth pattern on the envelope — the correction rises and falls with each vibrato cycle. Gap bridging fuses these dips into a single continuous tonal segment.

The trade-off: if `max_gap_ms` is too high, two distinct words with a short pause between them are fused. The gain applied to both is the average of the segment — useful if they have similar levels, problematic if one is a whisper and the other a belt. The user controls this via the slider (10–200 ms).

Pass 3 — minimum length filter + avg_rms:

Segments shorter than `min_seg_ms` (30 ms) are discarded. `avg_rms` is computed as the mean of `blocks[i].rms` across the segment.

10. Gain computation — `compute_gains`

Signature:

```

local gains = compute_gains(
    segments, blocks, target_db, macro_pct, per_block, smooth_alpha, gate_db
)

```

Gate threshold (v6)

Before any gain is computed, each block or segment is checked against `gate_db`:

```
-- Per-block mode:
if bl_db > gate_db then
    -- compute gain
end

-- else: gains[i] stays 1.0 (untouched)

-- Segment mode:
if seg_db > gate_db then
    -- compute gain for all blocks in segment
end
```

Purpose: prevents boosting breaths, microphone bleed, or background noise that briefly creeps above the adaptive silence threshold and gets misclassified as tonal. The gate threshold is applied at the gain stage (not the analysis stage), so changing it in the GUI recalculates the preview instantly without re-analysis.

Linear amplitude interpolation

Both modes use amplitude-domain interpolation rather than dB arithmetic:

```
local bl_lin      = db_to_lin(bl_db)      -- current level (linear)
local tgt_lin     = db_to_lin(target_db)  -- target level (linear)
local delta_lin   = (tgt_lin / bl_lin) - 1.0
local applied_lin = 1.0 + (delta_lin * macro_pct)
gains[i] = clamp(applied_lin, gain_min, gain_max)
```

Why not dB arithmetic: `(target_db - bl_db) * macro_pct` is numerically convenient but perceptually non-linear. At 50% strength, a +12 dB correction becomes +6 dB — but "+6 dB" does not perceptually sound like "half the correction". The linear amplitude interpolation is: at 0% nothing happens, at 100% the block reaches exactly the target, and at intermediate values the correction is proportional in perceived loudness.

Clamp bounds (pre-computed)

```
local gain_min = db_to_lin(-18)  -- max attenuation: prevents over-muting quiet blocks
local gain_max = db_to_lin(12)   -- max boost: prevents noise amplification
```

Computed once before the loops, not per-block.

Mode: segment gain (Melodyne — default)

A single constant gain per segment based on its `avg_rms`. The natural internal dynamics of each syllable — consonant attack, vowel sustain, tail — are preserved. This is the true Melodyne blob behaviour.

Mode: Volume Rider (per-block)

Each block gets its own gain based on its individual RMS. A double-pass zero-phase IIR smoothing filter is applied **within each segment** to prevent zipper noise:

```
-- Forward pass (left → right): smooths but introduces phase delay
local cur_g = gains[seg.from]
for i = seg.from + 1, seg.to do
    cur_g    = cur_g + alpha * (gains[i] - cur_g)
    gains[i] = cur_g
end

-- Backward pass (right → left): cancels the phase delay
cur_g = gains[seg.to]
for i = seg.to - 1, seg.from, -1 do
    cur_g    = cur_g + alpha * (gains[i] - cur_g)
    gains[i] = cur_g
end
```

Why segment-restricted: if the filter ran globally across the entire gains array, high gain from a loud syllable would bleed into adjacent silence blocks (`gains[i] = 1.0`), boosting breath noise audibly. Running it only within each tonal segment keeps non-tonal blocks at unity gain.

Zero-phase result: forward pass introduces group delay $-\phi(\omega)$; backward pass introduces $-\phi(-\omega) = +\phi(\omega)$, exactly cancelling it. The gain envelope is centred on the syllable.

11. Auto-target RMS

After each analysis, the target RMS slider is automatically set to the average RMS of tonal blocks only:

```

local tonal_rms_sum = 0
local n_tonal = 0
for _, bl in ipairs(gui.blocks) do
  if bl.is_tonal then
    n_tonal = n_tonal + 1
    tonal_rms_sum = tonal_rms_sum + bl.rms
  end
end

if n_tonal > 0 then
  local avg_tonal_db = lin_to_db(tonal_rms_sum / n_tonal)
  if avg_tonal_db > -60 then
    gui.auto_target_db = clamp(avg_tonal_db, -40, -6)
    gui.target_db = gui.auto_target_db
    save_settings(gui)
  end
end
end

```

Why tonal blocks only: the global RMS includes silences and noise that can pull the mean to -45 dB or lower. Targeting -45 dB would force a massive boost across the entire recording. Using only the tonal blocks captures the true natural level of the singing voice in this specific recording.

`gui.auto_target_db` is stored separately from `gui.target_db`, allowing the user to adjust the slider freely and restore the calculated value with the  **Auto** button:

```

if r.ImGui_SmallButton(ctx, "🔄 Auto") then
  gui.target_db = gui.auto_target_db
end

```

`auto_target_db` is reset to `nil` when the user clicks **Analyse** or **Change** — the button is hidden until a new analysis completes.

12. Envelope writing — `apply_take_envelope`

Take Envelope vs Track Envelope

The automation is written to the **Take Volume Envelope** — stored inside the item, moves with the item on the timeline.

Source validation (v6)

Before creating the accessor, three checks prevent silent failures:

```
local src = r.GetMediaItemTake_Source(take)
if not src then ... end

local filename = r.GetMediaSourceFileName(src, "")
if filename == "" then ... end -- file offline or missing

local src_type = r.GetMediaSourceType(src, "")
if src_type == "MIDI" or src_type == "EMPTY" then ... end
```

Envelope scaling mode

```
local env_mode = r.GetEnvelopeScalingMode(env)
local function to_env(g)
    return r.ScaleToEnvelopeMode(env_mode, g)
end
```

REAPER supports two envelope modes: amplitude (linear) and volume fader (dB). Raw linear values inserted without conversion produce incorrect results in fader mode and silently clip at the project ceiling (~+6 dB = 1.995 linear).

Envelope arming

After creation, the envelope's state chunk is checked and **ARM 1** is forced if absent or set to **ARM 0**. Without this, a newly created envelope may start in non-armed (Read-off) mode — automation is written but inaudible.

```
local _, env_str = r.GetEnvelopeStateChunk(env, "", false)
if env_str:find("ARM 0") then
    env_str = env_str:gsub("ARM 0", "ARM 1")
    r.SetEnvelopeStateChunk(env, env_str, false)
end
```

Intelligent decimation

Two thresholds decide whether a point is inserted:

Condition	Threshold	Meaning
<code>is_step</code>	<code>> 0.05</code> linear (≈ 0.4 dB)	Abrupt change at tonal/silence boundary
<code>is_drift</code>	<code>> 0.015</code> linear (≈ 0.1 dB)	Slow progressive change within a segment
<code>is_last</code>	always	Final block always anchored

Result: a few hundred points for a 3-minute take instead of 18 000.

Fade ramps

At `is_step` transitions, a pre-transition fade point prevents clicks:

```
local fade_start = max(
    env_t - (actual_fade * playrate),
    env_t_prev + (block_sec * 0.1 * playrate)
)
```

Clamp: `actual_fade = min(fade_sec, block_sec * 0.9)` — prevents the fade point from being placed before the previous block's timestamp, which would corrupt REAPER's envelope ordering.

Final return-to-unity point

```
local item_len      = r.GetMediaItemInfo_Value(item, "D_LENGTH")
local end_source_t = start_offs + (item_len * playrate)
r.InsertEnvelopePoint(env, end_source_t, to_env(1.0), 0, 0, false, true)
```

Without this point, REAPER extends the last inserted gain value to the end of the source file — a +4 dB gain would boost background noise after the last word indefinitely.

13. Cache system

Trig table cache

```
local trig_cache = {}
  -- key: "N_eff_n_bins"

local function get_trig_tables(N_eff, n_bins)
  local key = N_eff .. "_" .. n_bins
  if trig_cache[key] then return trig_cache[key] end
  -- compute cos[k][n] and sin[k][n] for k ∈ [1,n_bins], n ∈ [1,N_eff]
end
```

Eliminates ~200 million `math.cos` / `math.sin` calls for a 3-minute take.

Hann window cache

```
local hann_cache = {}
  -- key: N_eff (integer)
```

Cache invalidation

```
local function purge_caches()
  trig_cache = {}
  hann_cache = {}
end
```

Called when `block_ms` changes between analyses. Without purging, stale tables from a previous `N_eff` produce wrong DFT results silently. Lua's GC collects the old tables once no reference remains.

14. Settings persistence — ExtState

```
local EXT_SECTION = "VocalVolumeMacro_v6"

local function ext_load(key, default)
    local s = r.GetExtState(EXT_SECTION, key)
    return (s ~= "") and tonumber(s) or default
end

local function ext_save(key, val)
    r.SetExtState(EXT_SECTION, key, tostring(val), true) -- true = persist to disk
end
```

Persisted keys: `zcr_thresh`, `sc_thresh`, `flux_thresh`, `block_ms`, `target_db`, `gate_db`, `gap_ms`, `macro_pct`, `fade_ms`, `per_block`, `smooth_alpha`.

Save triggers: every slider/checkbox change + window close.

Load trigger: once at startup before `gui` is initialised.

freq_max_hz anti-clamp fix: on startup, `gui.freq_max_hz` would be 0, causing `clamp(sc_thresh, 200, max(0, 500)) = 500` to silently overwrite the saved 2500 Hz value. Fixed by computing `freq_max_hz` immediately when an item is selected (or using a 44100 Hz estimate when no item is present).

15. GUI loop and state machine

Window initialisation

```
r.ImGui_SetNextWindowSize(ctx, W, H, r.ImGui_Cond_FirstUseEver())
r.ImGui_SetNextWindowPos(ctx, 80, 60, r.ImGui_Cond_FirstUseEver())
```

`Cond_FirstUseEver` applies the default size/position only on the first ever launch. ImGui restores user-adjusted values on subsequent launches. `Cond_Once` was replaced because it could prevent resizing on some ReaImGui/OS combinations.

ImGui context management

```
-- r.ImGui_DestroyContext(ctx) ← must NOT be called in ReaImGui
-- The context is managed by Lua's garbage collector automatically.
```

In Dear ImGui C++, `DestroyContext` is required. In ReaImGui Lua, calling it manually causes a crash at window close — the context is freed by the GC when the script exits.

`vis = false` handling

```
if not vis then
    -- In ReaImGui, if Begin() returns false, End() must NOT be called.
    -- Calling End() here causes "Calling End() too many times!" and
    -- blocks the script from restarting.
    return open
end
```

GUI state machine

```
IDLE
| select item → freq_max_hz computed immediately
▼
ITEM_SELECTED
| click Analyse → locks item, auto_target_db = nil, stores timestamp
▼
ANALYSING — cancel → IDLE (accessor destroyed immediately)
|
| async.done = true
| → build_segments(gap_ms)
| → auto-target: gui.target_db = avg tonal RMS
▼
ANALYSED
| adjust gate/gap/target/strength → instant preview recalculation
| click Apply → guard: segments not empty
▼
APPLIED (single Ctrl+Z undo via Undo_BeginBlock/EndBlock)
| click Reset env. → unity gain, Ctrl+Z undoable
| click Change → unlocks item, clears analysis
▼
IDLE
```

Real-time parameters

Parameters that recalculate the preview without re-analysis:

Parameter	Mechanism
<code>target_db</code>	<code>compute_gains()</code> called with new target
<code>gate_db</code>	<code>compute_gains()</code> called with new gate
<code>gap_ms</code>	<code>build_segments()</code> + <code>compute_gains()</code> called
<code>macro_pct</code>	<code>compute_gains()</code> called
<code>per_block</code>	<code>compute_gains()</code> called
<code>smooth_alpha</code>	<code>compute_gains()</code> called
<code>fade_ms</code>	applied at write time only

16. Known limitations

Timestretched takes

`GetAudioAccessorSamples` reads the raw source file. The `D_PLAYRATE` sync fix handles constant-rate changes correctly. REAPER's élastique time-stretch preserves pitch during tempo changes — the source file is unchanged and the stretching is applied at playback. The accessor reads unstretched audio. Render the item to a new file before running the script if élastique stretch is active.

Gap bridging and word separation

`max_gap_ms` is a global parameter. For a take with both slow vibratos and fast phrasing, no single value is optimal. The slider provides control, but the trade-off cannot be eliminated — it's fundamental to any time-domain segmentation approach.

Very short takes

At `block_ms = 10`, a 50 ms take produces only 5 blocks. Double-pass smoothing across 5 values produces nearly identical gains throughout — per-block mode degrades to segment mode.

Envelope ceiling

`ScaleToEnvelopeMode` respects REAPER's configured ceiling (typically +6 dB $\approx$ 1.995 linear). The script's own `gain_max = db_to_lin(12)  $\approx$  3.98` can exceed this — REAPER's ceiling is the effective upper limit.

17. REAPER API reference

API call	Notes
<code>r.new_array(n)</code>	Mandatory for <code>GetAudioAccessorSamples</code> . Plain Lua table <code>{}</code> crashes or returns silence.
<code>r.GetAudioAccessorSamples(aa, sr, nch, t, bs, buf)</code>	<code>t</code> is source time (seconds from file start). Buffer is interleaved: <code>buf[(i-1)*nch+c+1]</code> .
<code>r.CreateTakeAudioAccessor(take)</code>	Decodes audio into RAM. Must be destroyed.
<code>r.DestroyAudioAccessor(aa)</code>	Set <code>async.aa = nil</code> immediately after to prevent double-free.
<code>r.GetMediaItemTakeInfo_Value(take, "D_STARTOFFS")</code>	Seconds from source file start to beginning of visible item.
<code>r.GetMediaItemTakeInfo_Value(take, "D_PLAYRATE")</code>	Take playback rate. Guard against 0 before using as divisor.
<code>r.GetMediaSourceFileName(src, "")</code>	Returns <code>""</code> for offline or missing files — use to validate before analysis.
<code>r.GetMediaSourceType(src, "")</code>	Returns <code>"MIDI"</code> , <code>"EMPTY"</code> , etc. — use to reject non-audio takes.
<code>r.GetTakeEnvelopeByName(take, "Volume")</code>	Returns <code>nil</code> if not yet created.
<code>r.Main_OnCommand(40693, 0)</code>	Toggles take volume envelope existence.
<code>r.GetEnvelopeScalingMode(env)</code>	Returns 0 (amplitude) or 1 (volume fader dB).
<code>r.ScaleToEnvelopeMode(mode, g)</code>	Converts linear gain to correct internal envelope value. Required to avoid clipping and mode mismatch.
<code>r.GetEnvelopeStateChunk(env, "", false)</code>	Read envelope state for ARM flag manipulation.
<code>r.SetEnvelopeStateChunk(env, str, false)</code>	Write modified state chunk back.
<code>r.DeleteEnvelopePointRange(env, 0, 1e12)</code>	Clears entire take envelope.
<code>r.InsertEnvelopePoint(env, t, v, 0, 0, false, true)</code>	Last arg <code>noSort=true</code> — defer sorting until all points inserted.
<code>r.Envelope_SortPoints(env)</code>	Required after mass insertion with <code>noSort=true</code> .
<code>r.GetExtState(section, key)</code>	Returns <code>""</code> if not set — always check before using.
<code>r.SetExtState(section, key, val, true)</code>	<code>true</code> = persist to disk across restarts.
<code>r.time_precise()</code>	High-resolution timer for ETA calculation.
<code>r.defer(fn)</code>	Schedules <code>fn</code> on next UI frame (~30 ms).

API call	Notes
<code>r.ImGui_Cond_FirstUseEver()</code>	Apply window size/pos only on first ever launch. Preferred over <code>Cond_Once</code> .
<code>r.ImGui_PushStyleColor</code> / <code>PopStyleColor</code>	Use for warning text — more theme-safe than <code>TextColored</code> with hard-coded colours.
<code>r.ImGui_TextWrapped</code>	Use for warning messages — prevents truncation when window is narrow.